Algorithms in Smart Speakers

Signal and Audio Processing

Algorithms in Smart Speakers

1. Adaptive Echo Cancellation (AEC)
2. Audio Prefiltering
3. Speaker Direction of Arrival and Beamforming
4. Voice Activity Detection
5. Wake phrase detection (Keyword Spotting)
6. Automatic Speech Recognition (ASR)
7. Natural Language Understanding and Responding
8. Text-To-Speech (TTS)

Adaptive Echo Cancellation

Setting: smart speaker outputs own sound (plays music, speaks, etc.)

Problem: this sound reflects from room walls and return to speaker's microphones

Goal: subtract from microphone audio signal the own output sound

LMS (Least Mean Square)

$x(n)$ - speaker's own sound

$d(n)$ - audio recorded by microphone

$w(n, i)$ - room impulse response at current time moment n

$$y(n) = \sum_{i=0}^{M-1} w(n,i)x(n-i) = \boldsymbol{w}(n)^T \boldsymbol{x}(n)$$ - predicted audio for microphone (assuming user is not speaking)

$e(n) = d(n) - y(n)$ - prediction error

$$J = e(n)^T e(n) \to \min_{\boldsymbol{w}(n)}$$

Gradient descent update step: $\boldsymbol{w}(n+1) = \boldsymbol{w}(n) - \mu \nabla_{\boldsymbol{w}} J = \boldsymbol{w}(n) + \mu e(n) \boldsymbol{x}(n)$

Problem: update is proportional to x(n), may lead to instabilities when x(n) is drastically changing

NLMS (Normalized LMS)

Idea: find minimal change to previous w(n), that can explain the current d(n)

$$\|\boldsymbol{w}(n+1) - \boldsymbol{w}(n)\|^2 \to \min_{\boldsymbol{w}(n+1)},$$

$$d(n) = \boldsymbol{w}(n+1)^T \boldsymbol{x}(n).$$

Using Lagrange function the optimal solution is:

$$\boldsymbol{w}(n+1) = \boldsymbol{w}(n) + \frac{e(n)}{\boldsymbol{x}(n)^T \boldsymbol{x}(n)} \boldsymbol{x}(n)$$

Practical formula: $\boldsymbol{w}(n+1) = \boldsymbol{w}(n) + \mu e(n) \boldsymbol{x}(n), \ \mu = \dfrac{\alpha}{\boldsymbol{x}(n)^T \boldsymbol{x}(n) + \varepsilon}$

In NLMS w(n) are updated on every time moment with large M. For acceleration Frequency Domain Adaptive Filter (FDAF) is used when updates are made by blocks in spectrum domain

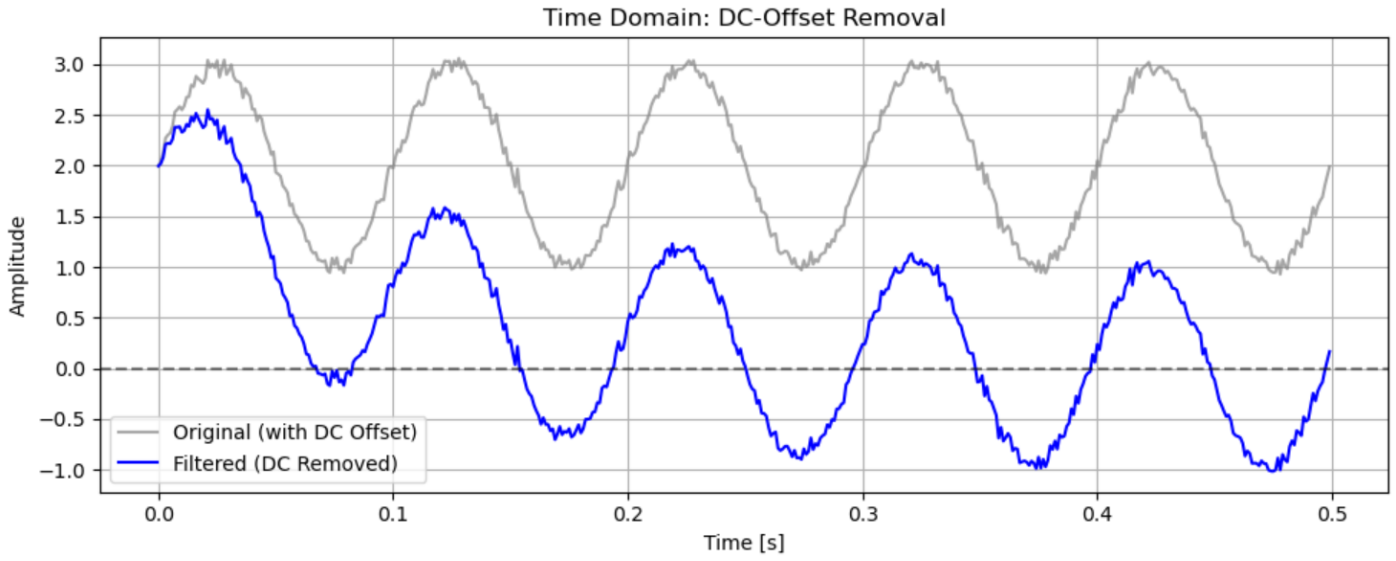
Audio Prefiltering

Goal: filter human input voice from all other noise sounds in the room (hum of refrigerators, keyboard clicking, dog barking, etc.)

Direct Current Offset Removal

Problem: microphone electronics can add constant shift to registered audio. This influences all other elements of audio processing

Solution: apply small IIR filter, that filters out low frequences below 5Hz



Direct Current Offset Removal

IIR filter: $y(n) = x(n) - x(n-1) - Ry(n-1),\ R = 0.99$

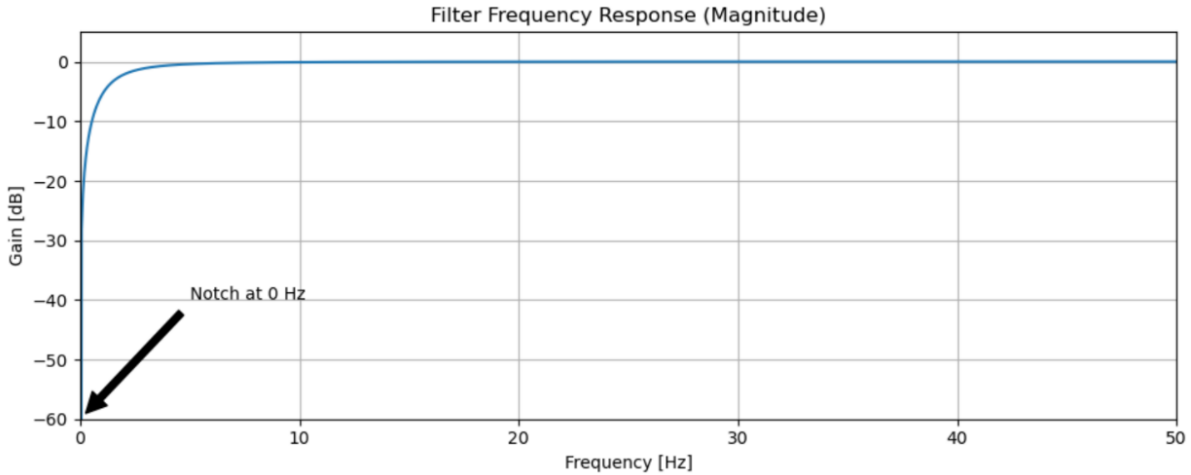
Z-transform: $X(z) = \sum_{n=-\infty}^{+\infty} x(n)z^{-n},\ z \in \mathbb{C}$

Transfer function: $H(z) = \frac{Y(z)}{X(z)} = \frac{1 - z^{-1}}{1 - Rz^{-1}}$

Zero z=0 – constant shift is fully cancelled

Pole z=R – filter is stable

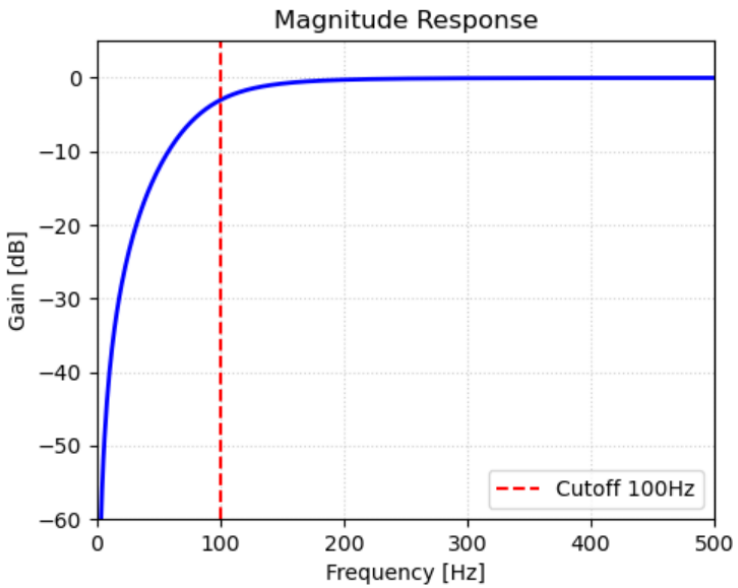
Magnitude response:

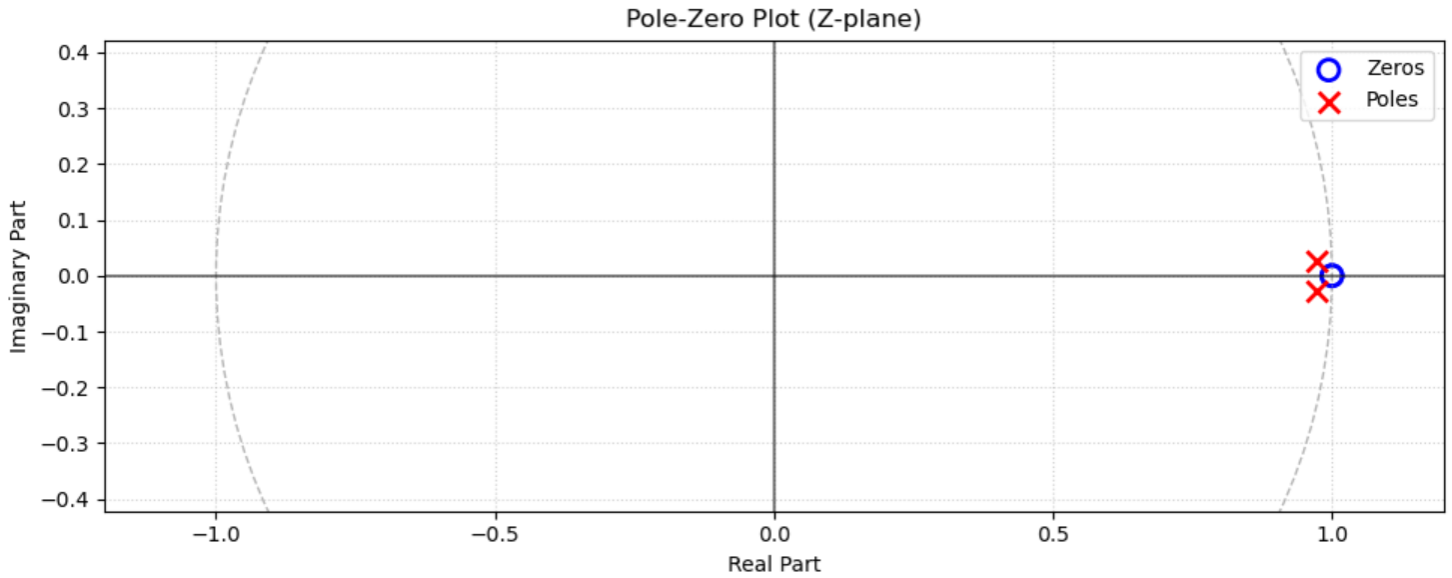


100 Hz High Pass Filter

Human voice has frequencies starting from 80-100 Hz. All lower frequencies are reasonable to filter out

Butterworth 2-nd order IIR filter: $H(z) = \dfrac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}$



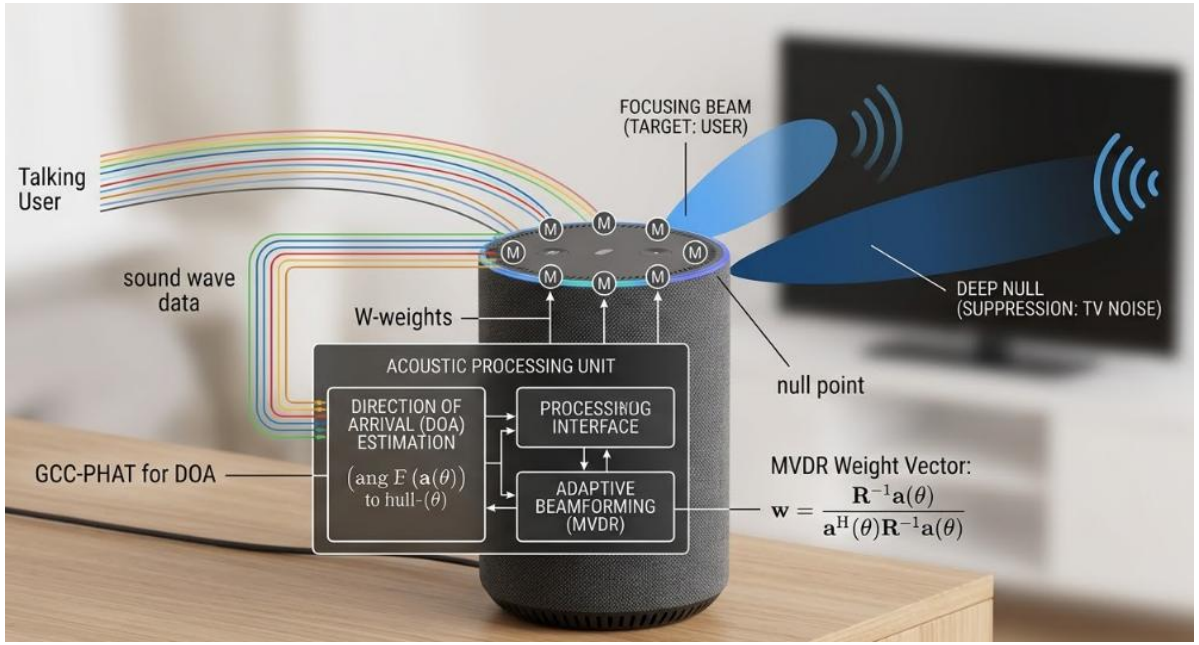


Microphone Array

Smart speaker has many microphones (usually located in a circle)

DC Removal, AEC, HPF are applied for each microphone separately

Next steps: determine angle of each voice source in room (Direction of Arrival) and amplify sound for direction from user / mute sound for direction from TV (Beamforming)

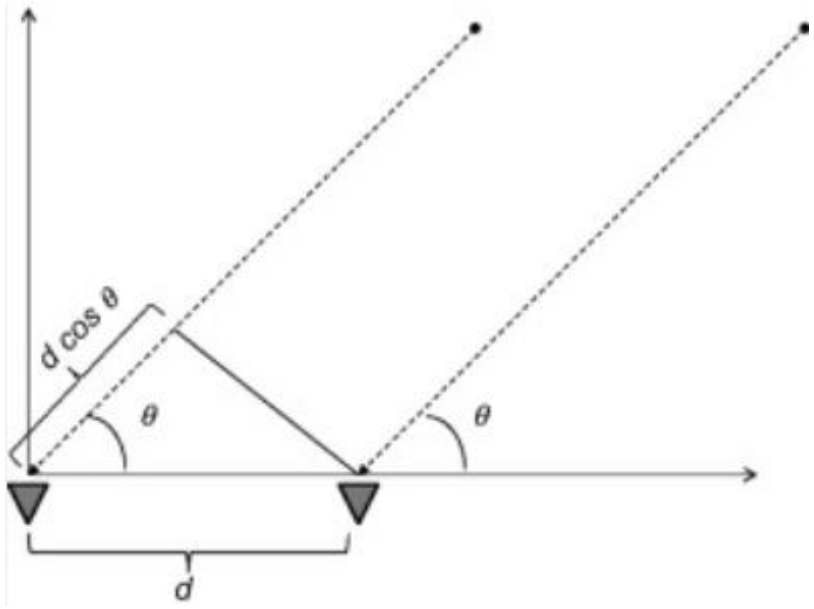


Direction of Arrival (DOA)

Consider microphone array located in a line

d - distance between two microphones

$\tau = \frac{d\cos(\theta)}{c}$ - delay in time (phase) for input signal on two microphones, c – sound speed



Find delay by maximizing cross-correlation:

$$R_{x_1x_2}(\tau) = \sum_n x_1(n)x_2(n-\tau) \to \max_\tau$$

Estimated angle: $\theta = \arccos\left(\frac{\tau c}{d}\right)$

GCC-PHAT for DOA (Generalized Cross-Correlation with PHAse Transform)

In rooms with strong echo, voice wall reflection can lead to false peaks in cross-correlation

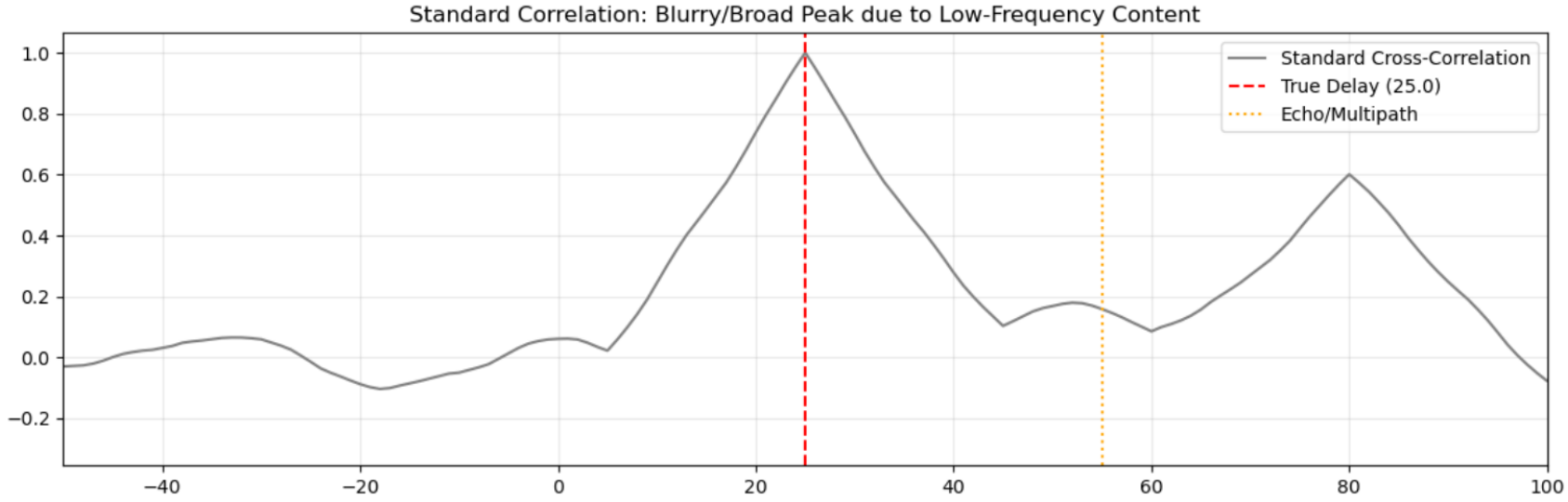
Idea: suppress information about amplitudes and consider only phases

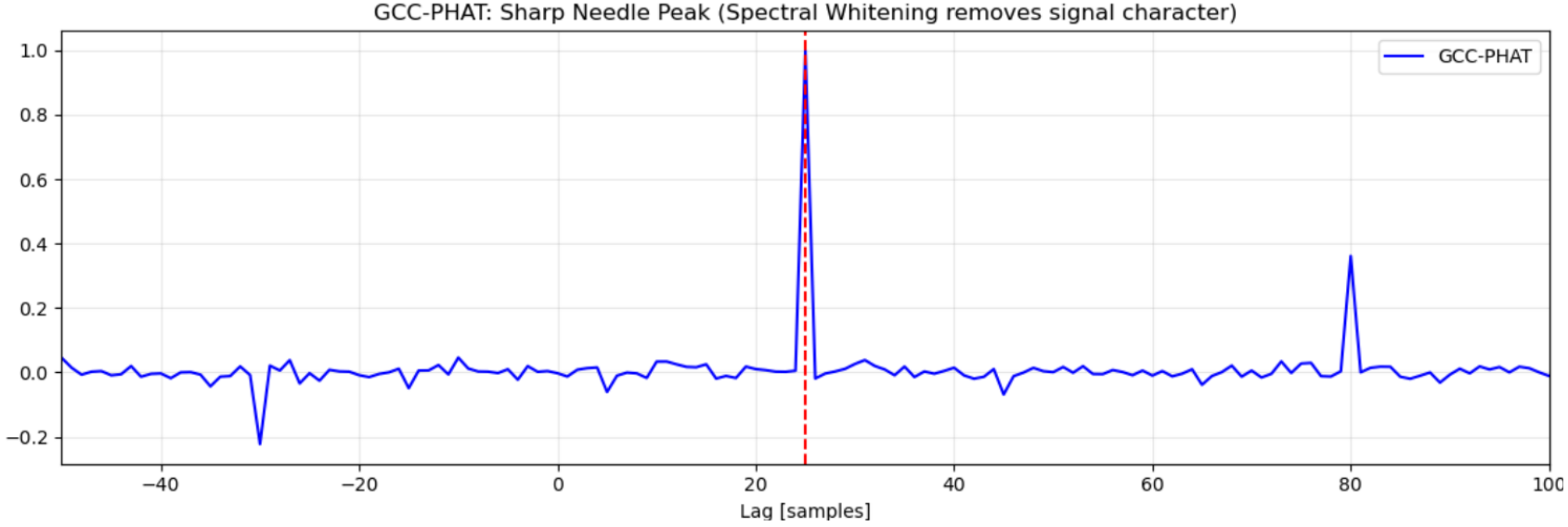
$$X_1(f) = FFT(x_1(n)),\ X_2(f) = FFT(x_2(n))$$

$$G_{x_1x_2}(f) = X_1(f)X_2(f)^* = |X_1(f)||X_2(f)| \exp(j(\theta_1(f) - \theta_2(f)))$$

$$R(\tau) = IFFT\left(\frac{X_1(f)X_2(f)^*}{|X_1(f)X_2(f)^*|}\right)$$

GCC-PHAT vs. Cross-Correlation





Delay-and-Sum Beamforming

Idea: artificially delay signals from microphones and sum

$$y(n) = \frac{1}{M} \sum_{i=0}^{M-1} x_m(n - \Delta_m)$$

In frequency domain:

$$Y(f) = \frac{1}{M} \sum_{i=0}^{M-1} X_m(f) \exp(-j2\pi f \Delta_m)$$

MVDR (Minimum Variance Distortionless Response) – more advanced Beamformer, that can amplify user signal and mute noise sources

Voice Activity Detection (VAD)

Goal: determine for given audio fragment: human voice vs something else

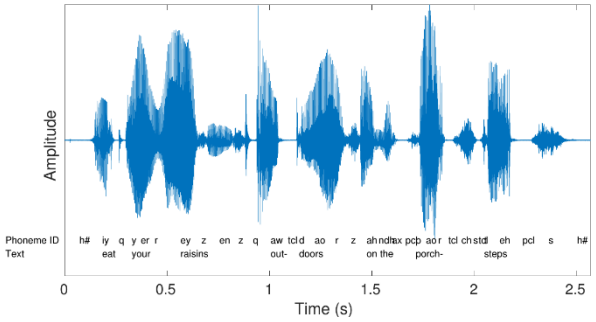
Energy for audio frame: $E = \sum_{n=1}^{N} x^2(n)$

Zero Crossing Rate: $ZCR = \frac{1}{2(N-1)} \sum_{n=1}^{N-1} |\mathrm{sgn}(x(n+1)) - \mathrm{sgn}(x(n))|$

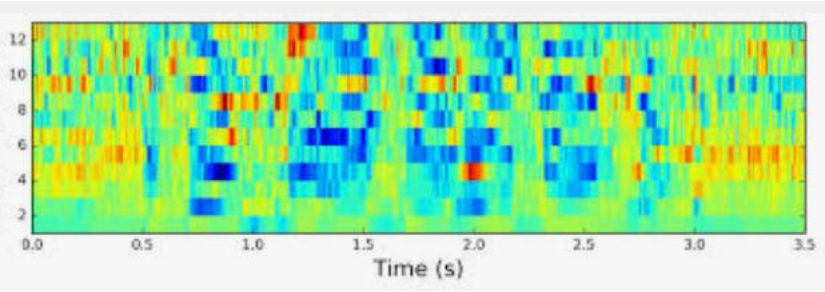
Spectral Flatness: $SF = \frac{\sqrt[n]{\prod_{n=0}^{N-1} |X(f_n)|^2}}{\frac{1}{N} \sum_{n=0}^{N-1} |X(f_n)|^2}$

VAD: If Energy is high, ZCR is low and SF is low then human voice

Neural Network based VAD



Audio waveform



MFCC Spectrogram
(or Log-Mel-Spectrogram)

NN

Class label (human / non-human)

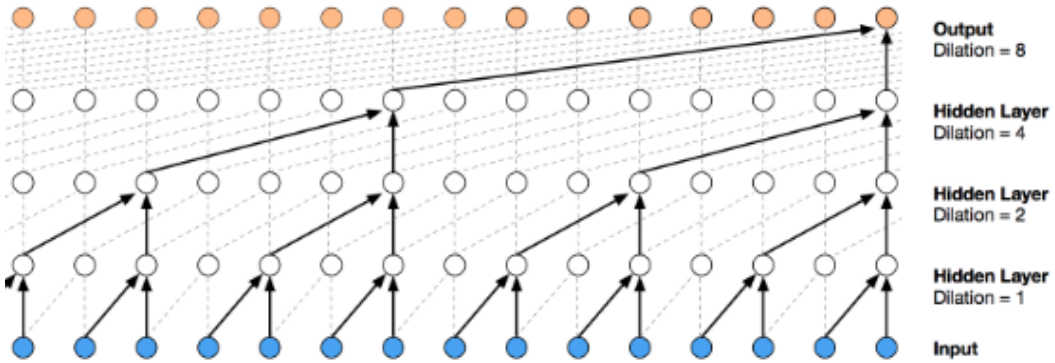
Neural Network based VAD

NN architecture: Time Convolutional Neural Network + (possibly) Recurrent Layers

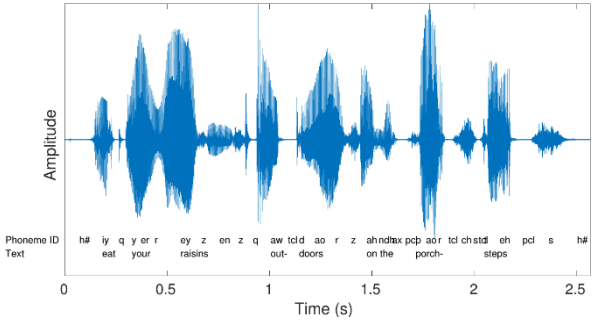
Training: classes 1) Human Voice, 2) Voice + Noise, 3) Voice + Music, 4) Music, 5) House devices, 6) outdoor noise, etc.

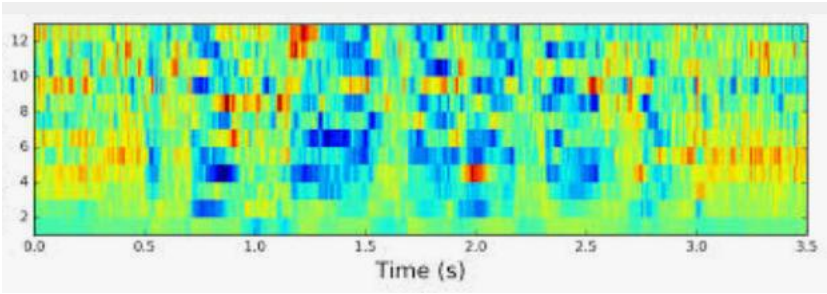
Testing: joint probability for all classes with Human Voice

Model example: Silero VAD



Key Word Spotting





NN

Class label („Alexa“/ something else)

Audio waveform

MFCC Spectrogram
(or Log-Mel-Spectrogram)

NN architecture: Temporal CNN + Recurrent

Learns to separate “Alexa” vs. “Election”, “A Lexus”, “Unleash her”, etc.

VAD + Key Word Spotting

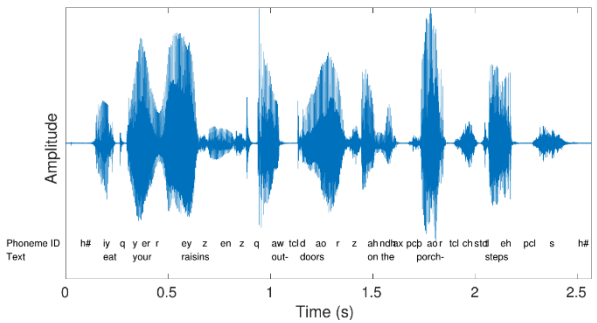
VAD is cheap and small network, applied for every frame

If VAD detects human voice, then local KWS network is asked

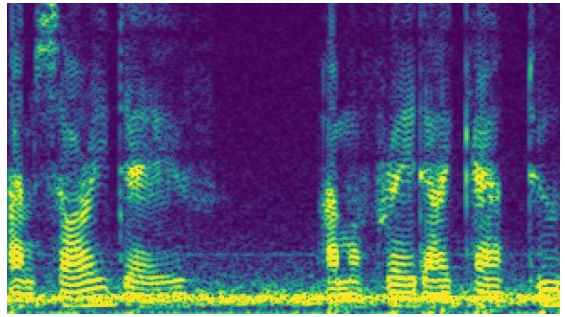
If local KWS detects keyword, then cloud connection is opened and heavy KWS network is asked. This network accounts also for TV advertisement

If global KWS detects keyword, then ASR (Automatic Speech Recognition) is started

ASR pipeline



Audio waveform



Log-Mel-Spectrogram

NN

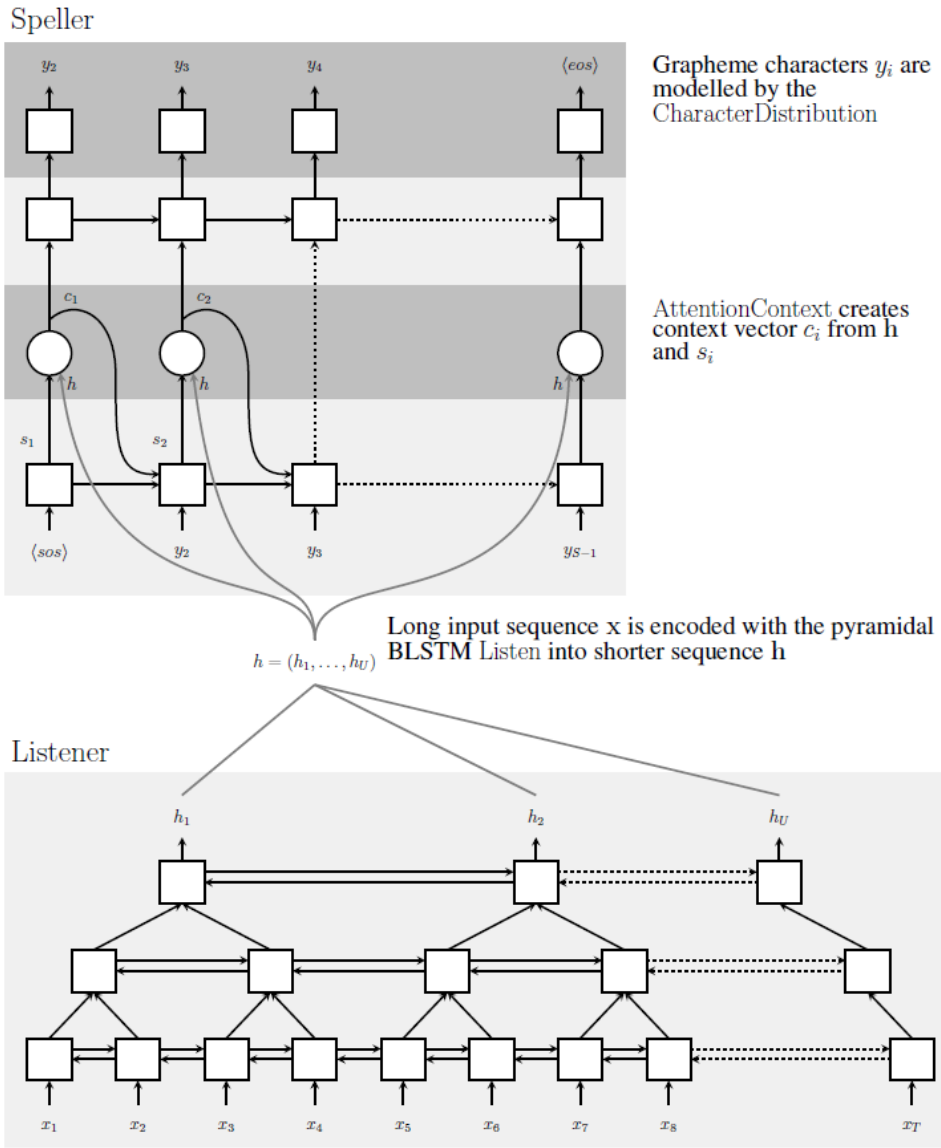
Hello, …

Output text

ASR architectures

- LAS: encoder-decoder model with attention
- DeepSpeech: encoder-only model with CTC loss
- RNN-Transducer: encoder-decoder model with monotonic alignment, suitable for streaming ASR

Listen, Attend and Spell (LAS, 2015)



Architecture: Encoder-Decoder model with attention

Encoder: pyramid bidirectional LSTM

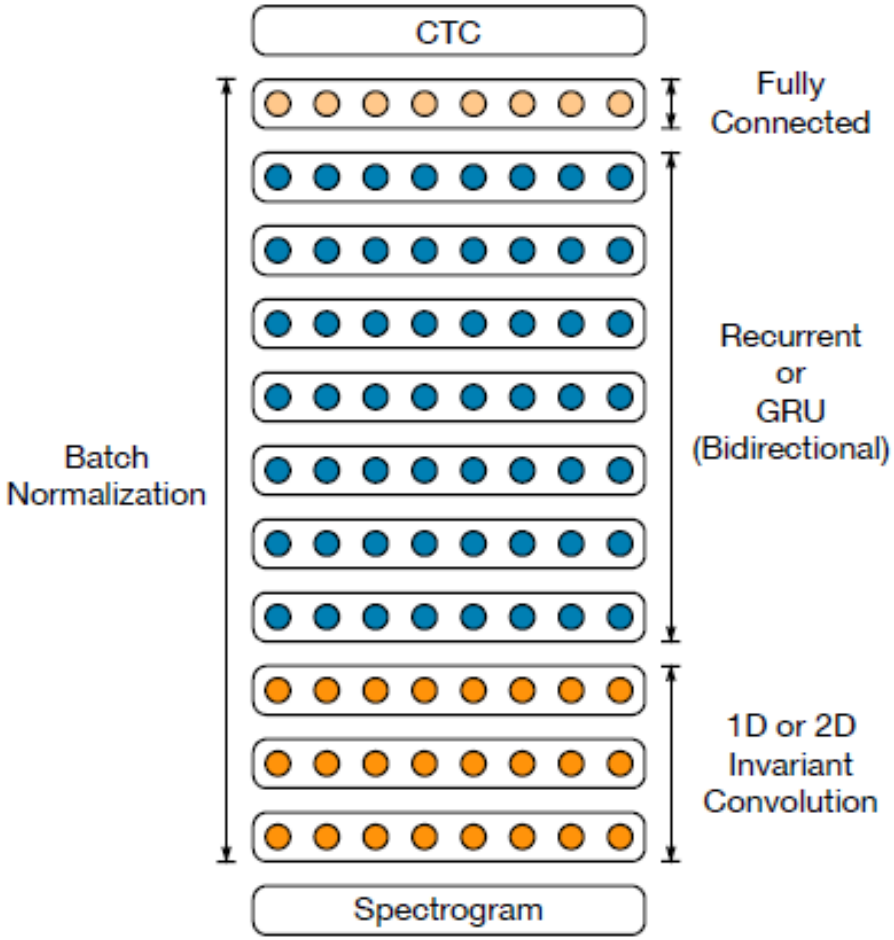
Decoder: autoregressive LSTM

Inference: Beam search

Pros: good quality, ability to use separately pretrained encoder for audio (Conformer-like) and decoder for text (GPT-like)

Cons: very slow on inference, requires full input audio to make recognition

Deep Speech 2 (2016)



Architecture: Encoder only model + CTC loss

Encoder uses 1) convolutional layers (1d or 2d), 2) RNN layers and 3) FC layers

Inference: beam search + separate language model (non-autoregressive word n-gram model like KenLM)

Pros: fast inference

Cons: requires separate language model, can't use pretrained text decoder

Architecture	Channels	Filter dimension	Stride	Regular Dev	Noisy Dev
1-layer 1D	1280	11	2	9.52	19.36
2-layer 1D	640, 640	5, 5	1, 2	9.67	19.21
3-layer 1D	512, 512, 512	5, 5, 5	1, 1, 2	9.20	20.22
1-layer 2D	32	41x11	2x2	8.94	16.22
2-layer 2D	32, 32	41x11, 21x11	2x2, 2x1	9.06	15.71
3-layer 2D	32, 32, 96	41x11, 21x11, 21x11	2x2, 2x1, 2x1	8.61	14.74

Using language models on inference

Example: I'll write it right now

Words "write", "right", "rite" have almost identical pronunciation

DeepSpeech: each output token is generated independently from each other => without separate language model can output "I'll right it right now"

LAS: decoder is autoregressive, next token is generated conditioned on previously generated token => can show good performance without additional language model

RNN Transducer (RNN-T, 2012)

Transducer – the best from two worlds (LAS and DeepSpeech): fast inference, can use pretrained models, suitable for streaming ASR

In ASR alignment between input audio and output text is monotonic

Audio Encoder: biLSTM or Conformer, for audio $x_1, \dots, x_T$ outputs hidden vectors $h_1, \dots, h_T$

Text decoder (prediction network): autoregressive LSTM or GPT, for text $y_1, \dots, y_U$ outputs hidden vectors $g_1, \dots, g_U$

Joiner network: small MLP, for each pair (h_t, g_u) outputs probabilities $p(y|h_t, g_u)$ for all text tokens and additional blank symbol $\emptyset$

RNN-T Inference

t - current audio position, u - current text position

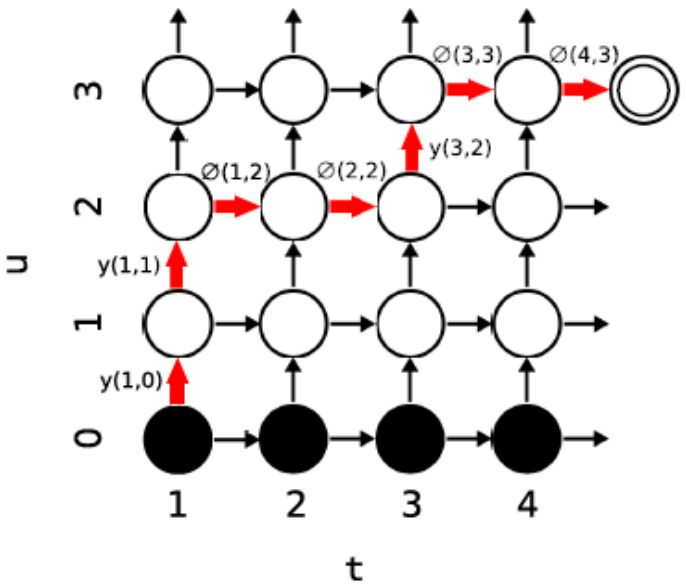
1. For current pair (t, u) compute probabilities $p(y \mid t, u)$ from Joiner network
2. If $y = \emptyset$ has the highest probability, then $t = t + 1$, “listen further”, go to step 1
3. If $y \neq \emptyset$ has the highest probability, then output this token, $u = u + 1$, go to step 1

RNN-T Training

Consider Drop operation: drop all empty symbols

$$\text{Drop}\{H, \emptyset, e, \emptyset, \emptyset, l, l, o\} = Hello$$

$$\text{Drop}\{\emptyset, H, \emptyset, e, \emptyset, l, \emptyset, l, o\} = Hello$$



Loss function for RNN-T is determined by probability:

$$p(y_{true} \mid x) = \sum_{y:\text{Drop}(y)=y_{true}} \prod_{u=0}^{U} p(y_u \mid x)$$

Efficient computation of this probability using Dynamic Programming:

$$\alpha(t, u) = \alpha(t-1, u)p(y = \emptyset \mid t, u) + \alpha(t, u-1)p(y = y_u \mid t, u)$$

RNN-T properties

For ASR streaming let’s process audio in chunks 300-500 ms

Prediction network is autoregressive => doesn’t need a separate language model on inference

Can use pretrained models for both audio encoder and text decoder

Text Normalization

Text normalization: transformation of written text to pronounced text

Example of input text: Hi, Dr. Smith! We've seen the report from NASA... it's about $19.99, or 20% off the usual price. I saw it on the news at 3:00 PM. The FBI might investigate, y'know? [laughter]

Example of normalized text: hi doctor smith we have seen the report from nasa it is about nineteen dollars and ninety nine cents or twenty percent off the usual price i saw it on the news at three p m the f b i might investigate you know

Elements of normalization: only lower case letters, no punctuation, numbers are pronounced, abbreviations are pronounced, special signs like $, % are pronounced, etc.

Inverse normalization operation is also needed

Text tokenization

Three options:

- **Separate letters:** a, b, c, … + blank, unknown; leads to very long text sequences

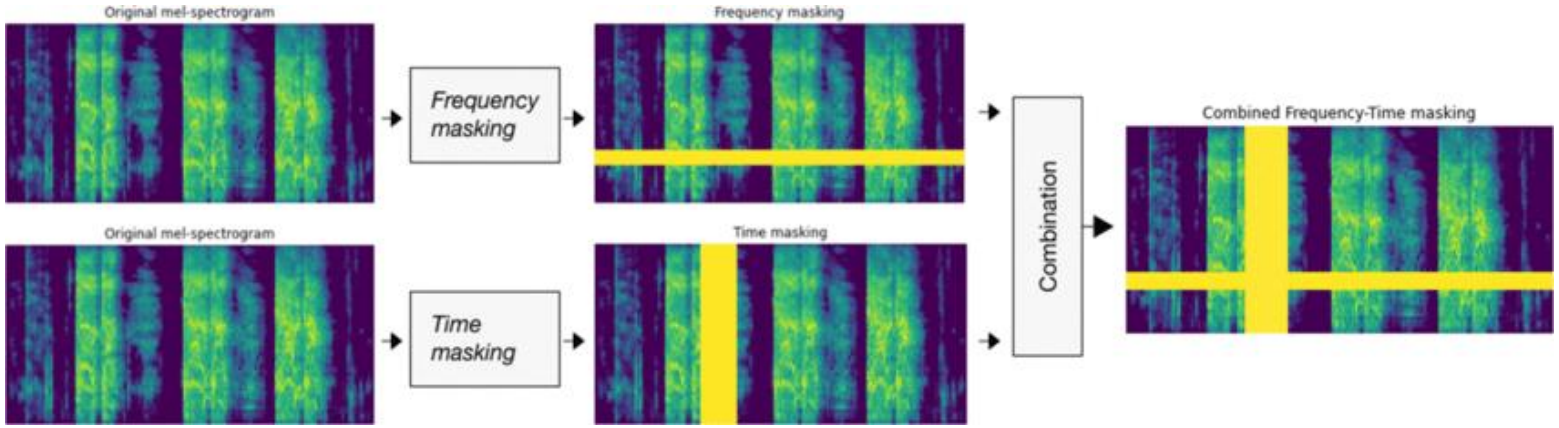
- **Phonemes:** hello → [h, ə, l, oʊ]; logical from linguistics point of view, requires non-trivial Grapheme-To-Phoneme (G2P) and inversed Phoneme-To-Grapheme models

  - The same letters may correspond to different pronunciation: tough, through, though
  - The same pronunciation may correspond to different words: write, right, rite, wright

- **Text tokens:** BPE, WordPiece, etc.; modern mostly used approach, also needed in case of pretrained text models

Audio augmentation

SpecAugment: Frequency and Time masking applied to spectrograms



Noise Injection: adding different noise to initial audio in time domain

Acoustic Simulation: adding filters of time domain audio in order to imitate big lecture hall, small bathroom, etc.

Time and Pitch distortion: accelerate and slow down audio signal, shifting pitch (F0)